Calibration d'une caméra

Séquence Vidéo – TP 1

Roméo Agostino – Master 2 VMI

1 Introduction

L'objectif de ce travail pratique est de réaliser la calibration d'une caméra à partir de plusieurs photographies d'une mire, pour estimer les paramètres intrinsèques de la caméra, caractériser la distorsion optique et déterminer la position et l'orientation d'un objet par rapport à la caméra.

Pour ce TP, j'ai utilisé la caméra de mon téléphone. La mire est un damier composé de 9×7 cases, soit $8 \times 6 = 48$ coins intérieurs détectables.

N'ayant pas de damier, j'ai affiché la mire sur l'écran d'un ordinateur portable. J'ai réalisé 8 photographies avec différentes positions et orientations de la caméra, pour varier l'observation.

Les images utilisées ont une résolution de 1536×1152 pixels et la taille d'une case de la mire a été estimée à environ 2 cm.

2 Méthode

Les images sont chargées avec `cv2.imread()`, converties en niveaux de gris, puis les 48 coins intérieurs sont détectés avec `findChessboardCorners()` et affinés avec `cornerSubPix()`.

Pour chaque photographie, `objpoints` contient les coordonnées des coins sur le damier et `imgpoints` leurs coordonnées en pixels. Les 8 photographies ont été utilisées pour la calibration avec `calibrateCamera()`.

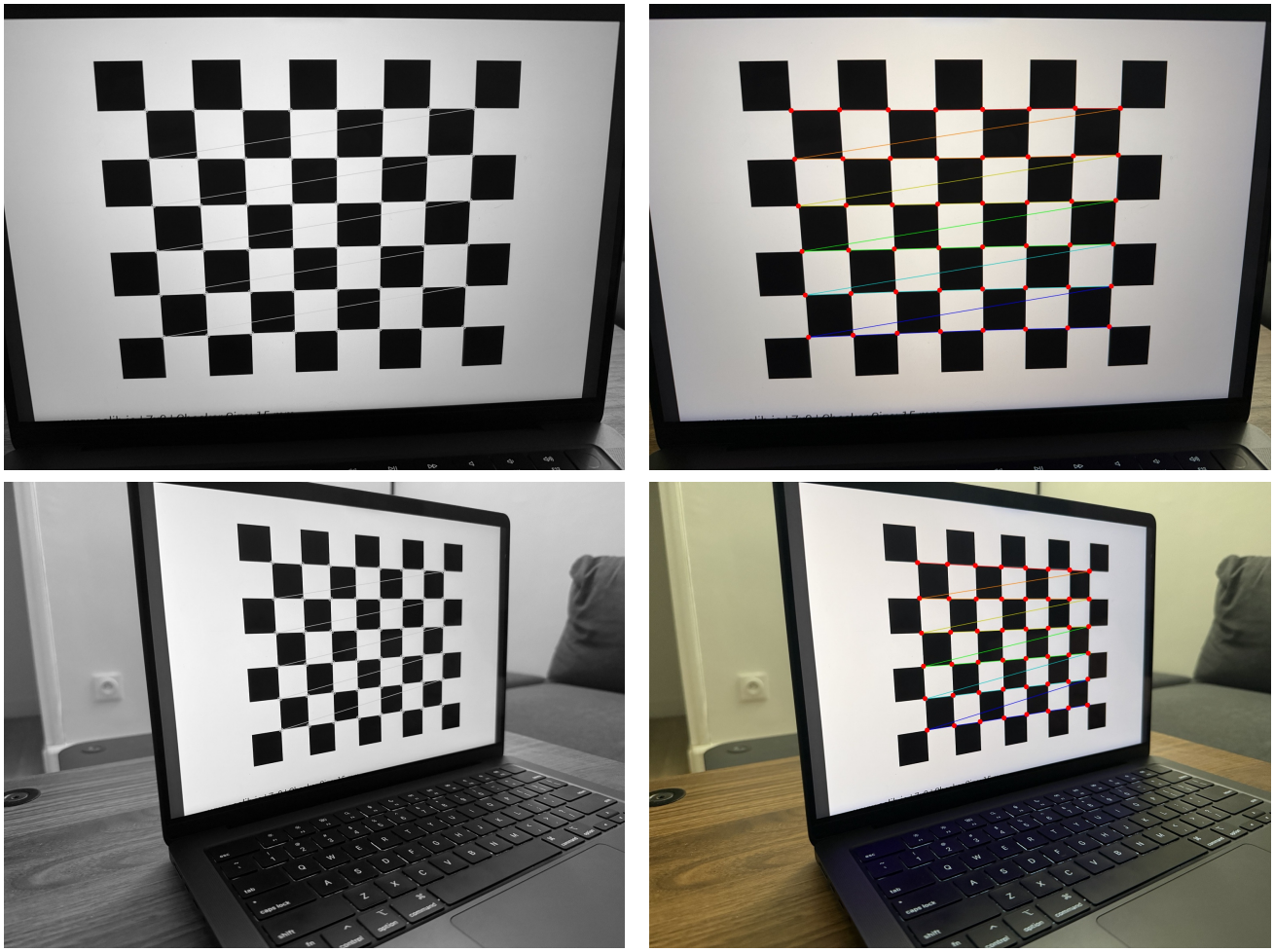


Figure 1 – Détection des 48 coins intérieurs sur plusieurs photographies

3 Résultats de calibration

La matrice intrinsèque estimée est :

$$K = \begin{pmatrix} 1104{,}62 & 0 & 769{,}64 \\ 0 & 1104{,}24 & 580{,}68 \\ 0 & 0 & 1 \end{pmatrix}.$$

Les focales f_x et f_y sont exprimées en pixels. Les deux focales sont très proches et le point principal est voisin du centre de l'image.

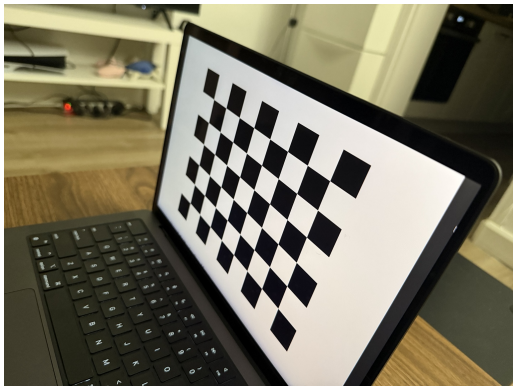
Paramètre	Valeur
Erreur RMS de reprojection	0,177 pixel
Distorsion radiale k_1, k_2, k_3	0,1783 , −0,3943 , −0,6383
Distorsion tangentielle p_1, p_2	0,000298 , −0,001165
Dimensions de capteur utilisées (estimées)	9,80 × 7,30 mm
Focale physique estimée	7,048 mm ≈ 0,00705 m
Champ de vision horizontal / vertical	69,62° / 55,09°

TABLE 1 – Valeurs obtenues avec `calibrateCamera()` et `calibrationMatrixValues()`.

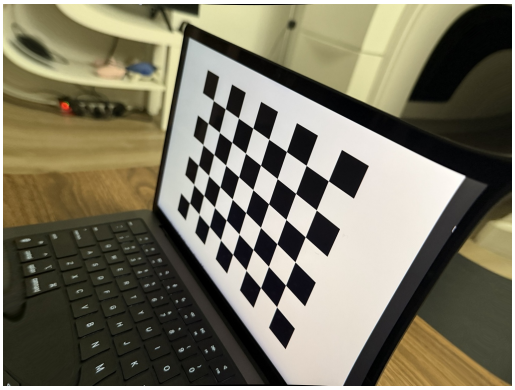
L'erreur RMS de 0,177 pixel indique un faible écart moyen entre les coins détectés et les points reprojetés. La focale en mètres a été estimée avec `calibrationMatrixValues()` : elle dépend des dimensions de capteur supposées, et n'est donc pas une mesure physique exacte.

4 Correction de la distorsion

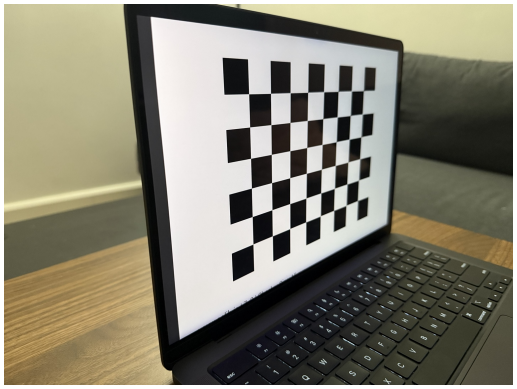
La fonction `undistort()` permet de corriger les déformations optiques à partir des paramètres calculés. Elle ne supprime pas l'effet de perspective dû à l'inclinaison du damier.



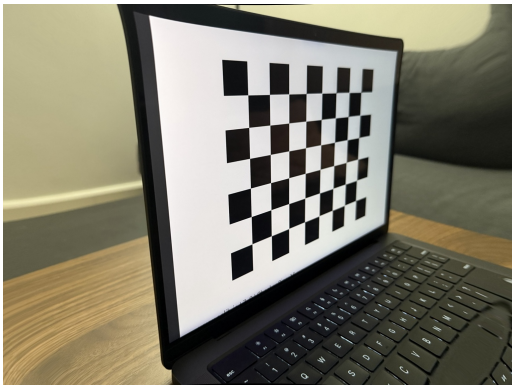
(a) Image originale



(b) Après correction.



(c) Image originale



(d) Après correction.

FIGURE 2 – Comparaison avant/après la correction de distorsion.

5 Estimation de pose et cube 3D

`solvePnP()` utilise les correspondances entre coins 3D et 2D pour estimer la rotation (`rvec`) et la translation (`tvec`) du damier dans le repère de la caméra. Les axes sont affichés avec `drawFrameAxes()`. L'origine (0,0,0) est placée sur le premier coin intérieur du damier.

Rotation (rvec) :

$$rvec = \begin{pmatrix} 0{,}289 \\ 0{,}864 \\ 0{,}470 \end{pmatrix}$$

Translation (tvec) :

$$tvec = \begin{pmatrix} -1{,}851 \\ -3{,}987 \\ 15{,}256 \end{pmatrix}$$

Pour aller un peu plus loin, j'ai construit les 8 sommets d'un cube en 3D. La fonction `projectPoints()` projette ces sommets sur la photographie, puis `line()` permet de dessiner les douze arêtes en couleur.

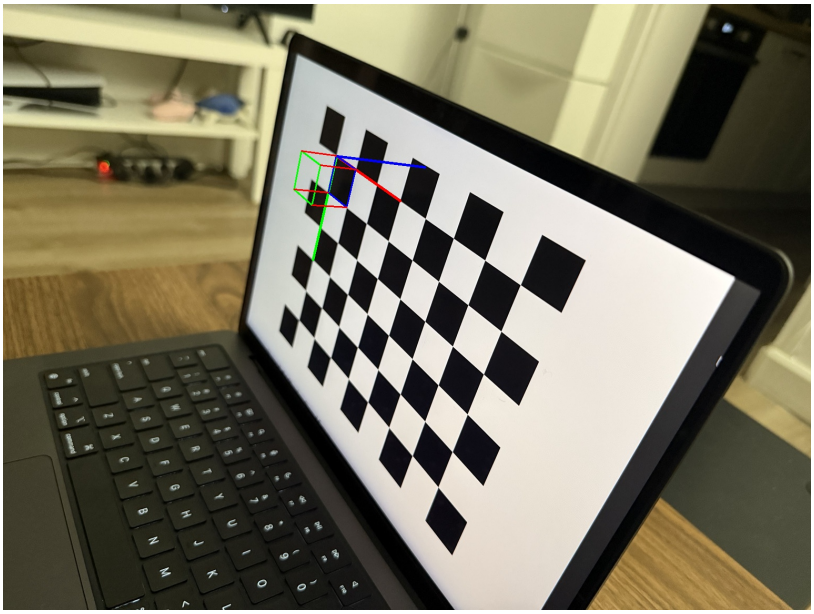


FIGURE 3 – Axes du repère et cube 3D projeté sur la mire.